🧠 四、AI 原型图纸 (AI-Generated Ideal Architecture Blueprint)

📌 文档定位

本图纸由大模型基于当前项目业务规则（命令驱动文本游戏）独立生成，旨在构建一个**高内聚、低耦合、可扩展**的理想架构蓝图，用于指导后续 Sprint 的结构演进。

该架构不依赖当前实现细节，而是从“职责分离”和“长期演进能力”出发设计。

🏗️ 一、总体分层架构（To-Be Architecture）

系统被划分为五个核心层级：

1. Transport Layer（输入传输层）

→ 输入/输出通道（CLI / Web / API）

⬇️

2. Application Layer（应用编排层）

→ 用例编排（Use Case）

⬇️

3. Domain Layer（领域层）

→ 核心业务模型（Player / Room / World）

⬇️

4. Presentation Layer（表现层）

→ 输出渲染（文本 / 消息协议）

⬇️

5. Infrastructure Layer（基础设施层）

→ 日志 / 存储 / 网络 / 配置

🔍 二、各层职责说明

1️⃣ Transport Layer（输入传输层）

职责：

- 接收用户输入（CLI / Web / Socket）
- 管理会话生命周期（启动 / 退出 / 中断）
- 将原始输入转交给 Application 层

典型组件：

- CLIHandler（终端输入处理）
- SessionManager（会话控制）

设计原则：

- 不包含业务逻辑
- 不直接调用领域对象

2️⃣ Application Layer（应用编排层）

职责：

- 解析命令（Command Routing）
- 编排业务流程（Use Case）
- 控制调用顺序（验证 → 执行 → 返回结果）

典型结构：
CommandHandler
↓

Command Registry（命令注册表）
↓
UseCase（如 MovePlayerUseCase）

设计原则：

- 只做“流程控制”，不做具体实现
- 不关心输入来源，也不关心输出格式

3️⃣ Domain Layer（领域核心层）

职责：

- 封装核心业务规则与状态
- 表达游戏世界的真实模型

核心实体：

- Player（玩家）
- Room（房间）
- World（地图）
- （扩展）Combat / Item / NPC

示例：
Player.move(direction)
Room.getNeighbor(direction)
World.getRoom(id)

设计原则：

- 不依赖 UI / 控制台
- 不包含 I/O 操作
- 高内聚：行为属于对象本身

4️⃣ Presentation Layer（表现层）

职责：

- 将业务结果转换为用户可读内容
- 统一输出格式（文本 / JSON / 消息码）

典型结构：
Result → Presenter → 输出文本

示例：
{ type: "MOVE_SUCCESS", payload: {...} }
→ "You moved to the north room."

设计原则：

- 统一输出协议
- 支持未来多端（CLI / Web）

5️⃣ Infrastructure Layer（基础设施层）

职责：

- 提供技术支撑能力

包括：

- Logger（日志）
- Storage（存档/数据库）
- Network（未来联机支持）
- Config（配置管理）

设计原则：

- 可替换
- 不影响上层业务逻辑

🔗 三、核心调用链（理想流程）

用户输入（CLI）
↓
Transport Layer（读取输入）

↓
Application Layer（解析命令）
↓
UseCase（执行业务流程）
↓
Domain Layer（更新状态）
↓
返回 Result（结构化结果）
↓
Presentation Layer（渲染输出）
↓
输出到终端

⚙️ 四、关键设计模式

1. Command Pattern（命令模式）

- 每个命令独立为对象（look / go / quit）
- 替代 switch-case

2. Use Case Pattern（用例模式）

- 每个业务流程独立
- 如：
  - MovePlayerUseCase
  - LookRoomUseCase

3. Result / Error Model（统一返回模型）

统一结构：
{
success: true/false,
code: "ERR_INVALID_DIRECTION",

message: "...",
data: {}
}

4. Dependency Inversion（依赖反转）

- 高层不依赖低层实现
- 通过接口解耦

🚀 五、架构优势总结

相比当前实现，该理想架构具备以下优势：

1. **高内聚**：业务逻辑集中在 Domain 层
2. **低耦合**：输入、业务、输出完全解耦
3. **可扩展**：新增功能（战斗/NPC）无需修改核心流程
4. **可测试**：每层均可独立单测
5. **多端支持**：CLI → Web → API 平滑演进

📌 六、适配项目未来演进

该架构可直接支持：

- Sprint 2：战斗系统 / 物品系统
- Sprint 3：多人联机（WebSocket）
- Sprint 4：前端 UI 接入（React / Web）

🧩 七、总结

AI 原型图纸提供的是一种**结构上限（Upper Bound）**：

- 当前代码：偏“流程驱动 + 快速实现”

- 理想架构：偏“分层清晰 + 可长期演进”

本图纸将作为后续重构的参考蓝图，而非一次性强制落地目标。